

Classification Models

COMP6246: Machine Learning Technologies (MSc)

Dr Indu Bodala (i.p.bodala@soton.ac.uk)

28 October 2026

Lecture Outline

- By the end of this lecture, you will be able to
 - Understand how Classification models work
 - Identify different types of classification problems – linear and non-linear
 - Understand Logistic Regression and Support Vector Machines

Classification

- A method where each data point is predicted to belong to a particular class or not
- Example: classify whether a handwritten digit is 5 (i.e., a binary classifier)
- We start with designing a *decision function* and a *decision threshold*
 - Calculate the functional value of the input using the *decision function*
 - Compare this value to the *decision threshold* to determine which class it belongs to



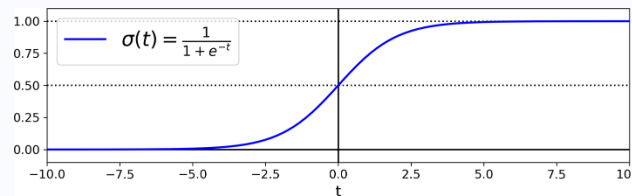
Digits from MNIST dataset

Logistic Regression – Estimating Probabilities

- Some regression algorithms can be used for classification tasks
- *Logistic Regression* (also called *Logit Regression*) estimates the probability that an instance belongs to a particular class

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^T \mathbf{x})$$

- $\sigma(\cdot)$ —is a *sigmoid function* (i.e., S-shaped) that outputs a number between 0 and 1.



- If $\hat{p} > 0.5$, the the model predicts that the instance belongs to that class, i.e. predicted label is 1
- If $\hat{p} < 0.5$, the the model predicts that the instance *doesn't* belongs to that class, i.e. predicted label is 0

Logistic Regression – Training

- We need to set the parameter vector θ so that the model estimates high probabilities for positive instances ($y = 1$) and low probabilities for negative instances ($y = 0$).

$$c(\theta) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1 - \hat{p}) & \text{if } y = 0 \end{cases}$$

- For the whole training set, the cost function can be written as

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})]$$

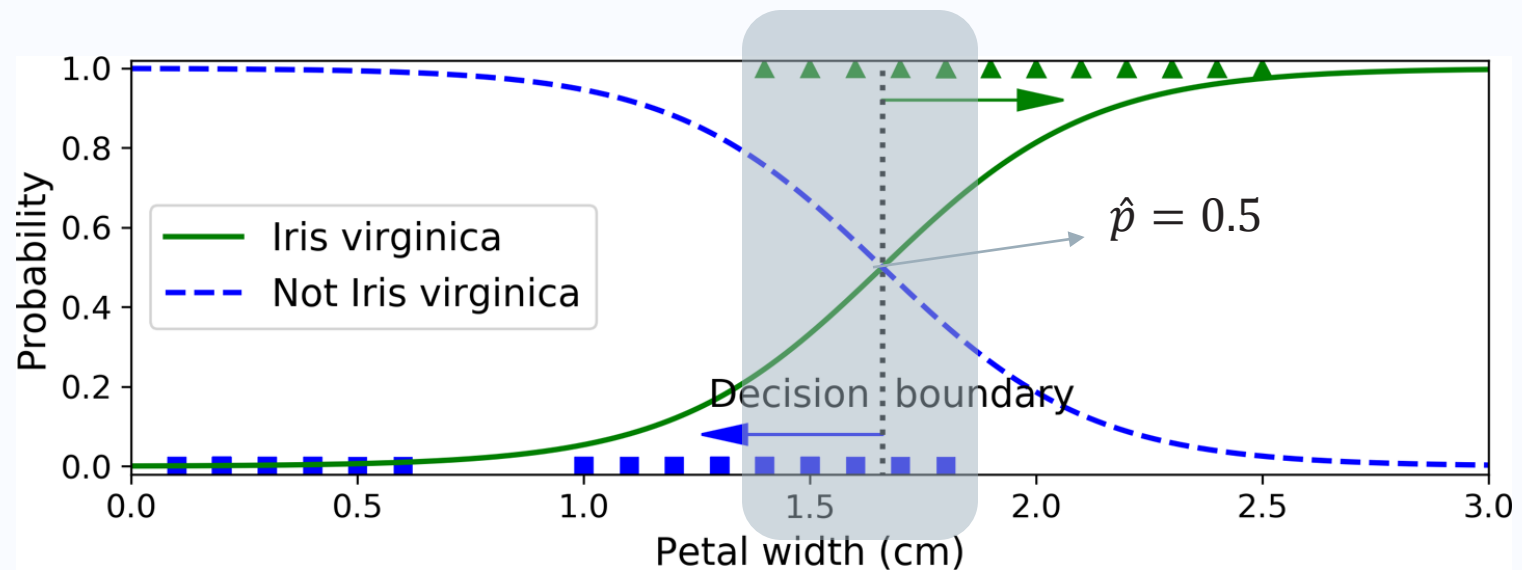
- And the derivative is

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m (\sigma(\theta^T \mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

- No known closed form solution, but is a convex function
- We can use Gradient descent optimization

Decision Boundary

- Example task – Classifying whether a flower is Iris Virginica or not based on the petal width
- Overlapping values of the petal width between the two classes
 - The classifier is uncertain in this region
 - The classifier will return whichever class is most likely



Softmax Regression

- Softmax or Multinomial Logistic Regression
 - Supports multi-class classification directly without combining Binary classifiers
- Softmax Regression computes a score of $s_k(\mathbf{x})$ for each class k and then estimates the probability of each class by applying a *softmax function* (or normalized exponential) to the scores

$$s_k(\mathbf{x}) = (\boldsymbol{\theta}^{(k)})^\top \mathbf{x} \longrightarrow \text{Each class has its own parameter vector}$$

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

$$\hat{y} = \operatorname{argmax}_k \sigma(\mathbf{s}(\mathbf{x}))_k = \operatorname{argmax}_k s_k(\mathbf{x}) = \operatorname{argmax}_k (\boldsymbol{\theta}^{(k)})^\top \mathbf{x}$$

The *argmax* operator returns the value of a variable that maximizes a function.

Cross Entropy Cost Function

- We use Cross Entropy cost function

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \left(\sum_{k=1}^K y_k^{(i)} \log \hat{p}_k^{(i)} \right)$$

For $k = 2$, this becomes
a logistic regression

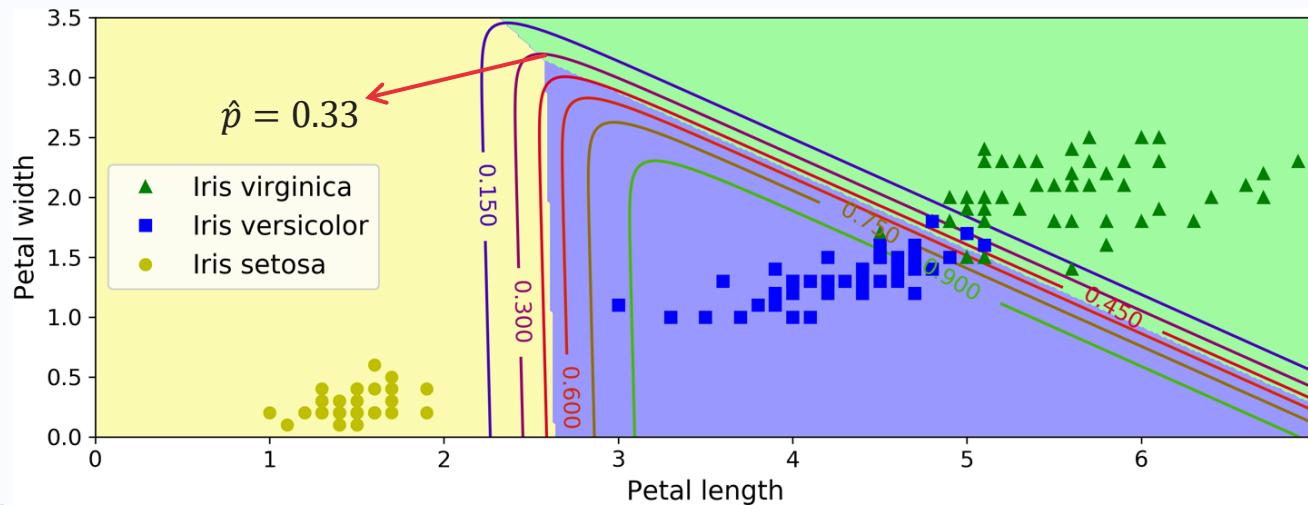
- Gradient vector

$$\nabla_{\theta^{(k)}} J(\Theta) = \frac{1}{m} \sum_{i=1}^m \left(\hat{p}_k^{(i)} - y_k^{(i)} \right) \mathbf{x}^{(i)}$$

- Use Gradient Descent to find the parameter matrix Θ that minimizes the cost function.

Softmax Regression – Decision Boundary

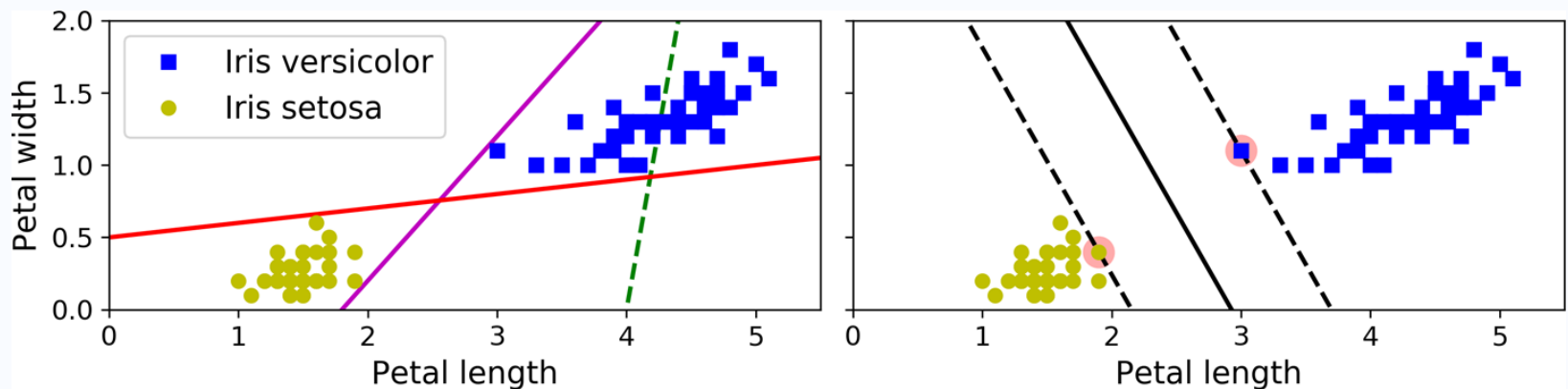
- Example task – classifying 3 types of Iris flowers – Iris virginica, Iris versicolor and Iris setosa based on petal length and petal width



- Note
 - Decision boundaries between two classes is still linear
 - At the point where all decision boundaries meet, all classes have an equal estimated probability of 33%

Support Vector Machines

- Can perform linear or non-linear classification
- Well suited for complex small or medium datasets

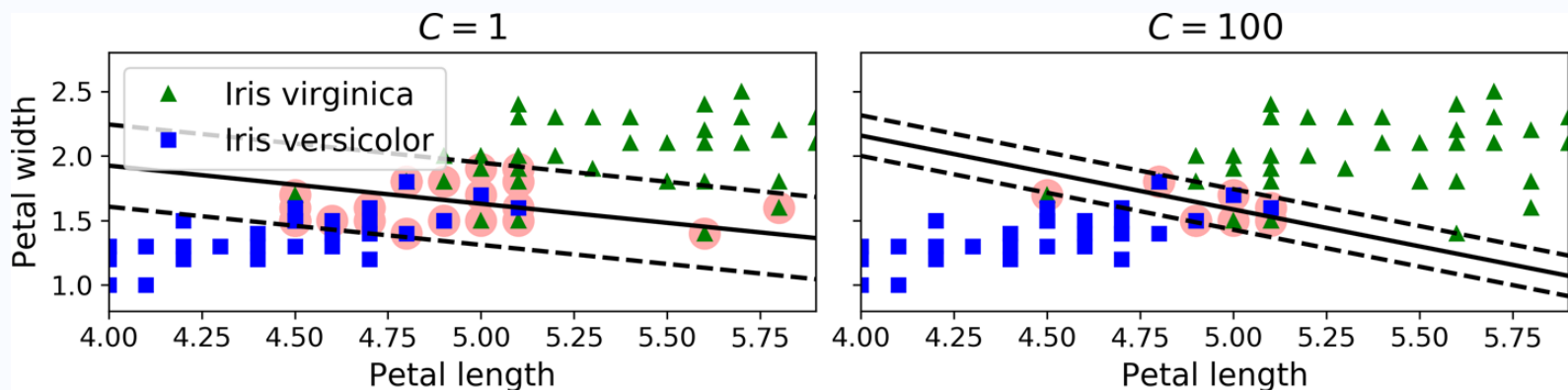
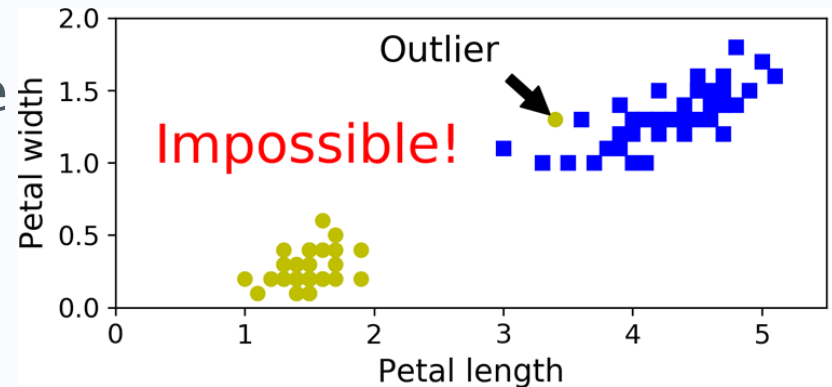


Large margin classification

- You can think of an SVM classifier as fitting the widest possible street (represented by the parallel dashed lines) between the classes

Soft Margin Classification

- Hard margin works only if the data is linearly separable
- The solution is to have a flexible model that allows for *margin violations*
- The objective is to find a good balance between keeping the margin as large as possible and limiting the *margin violations*.



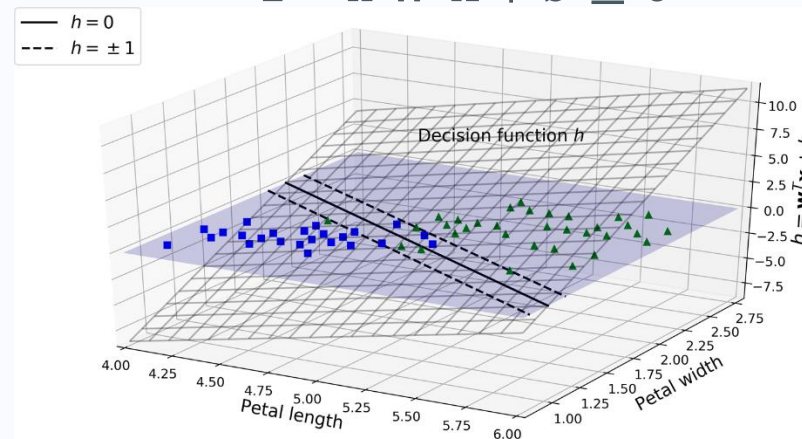
Linear SVM prediction

- Decision function

$$\mathbf{w}^T \mathbf{x} + b = w_1 x_1 + \dots + w_n x_n + b$$

- SVM classifier prediction

$$\hat{y} = \begin{cases} 0 & \text{if } \mathbf{w}^T \mathbf{x} + b < 0, \\ 1 & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0 \end{cases}$$



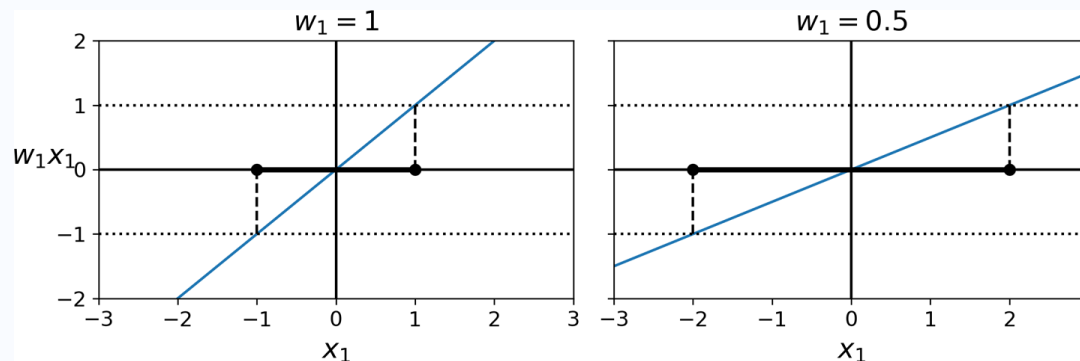
- At the decision boundary, we have $h = 0$ and at the margins, by convention, we have $h = \pm 1$

Linear SVM – Training

- SVM classifier prediction

$$\hat{y} = \begin{cases} 0 & \text{if } \mathbf{w}^\top \mathbf{x} + b < 0, \\ 1 & \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0 \end{cases}$$

- Objective: Find $\mathbf{w}$ and b that (1) makes the margin as wide as possible while (2) avoiding (or limiting) margin violations
- Slope* of the decision function = $\|\mathbf{w}\|$



- Smaller weight vector results in larger margin
- We want to minimize $\|\mathbf{w}\|$ to get a large margin

Hard-margin SVM Objective

- To avoid margin violations, we need the decision function to be greater than 1 for all positive training instances and lower than -1 for negative training instances.
- We define $t^{(i)} = -1$ for negative instances (if $y^{(i)} = 0$) and $t^{(i)} = 1$ for positive instances (if $y^{(i)} = 1$)
- Then, $t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1$ for all instances
- Hard-margin SVM classifier objective

$$\begin{aligned} & \underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2} \mathbf{w}^\top \mathbf{w} \\ & \text{subject to } t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 \text{ for } i = 1, 2, \dots, m \end{aligned}$$

Soft-margin SVM Objective

- For the soft margin SVM, we introduce a *slack variable* $\zeta^{(i)} \geq 0$ for each instance $\rightarrow$ measures how much violation is allowed for each instance
- We have two conflicting objectives:
 - Make the slack variables as small as possible to reduce the margin violations
 - Make $\frac{1}{2} \mathbf{w}^\top \mathbf{w}$ as small as possible to increase the margin
 - Hyperparameter C is used to define the tradeoff
- Soft margin SVM classifier objective

$$\underset{\mathbf{w}, b, \zeta}{\text{minimize}} \quad \frac{1}{2} \mathbf{w}^\top \mathbf{w} + C \sum_{i=1}^m \zeta^{(i)}$$

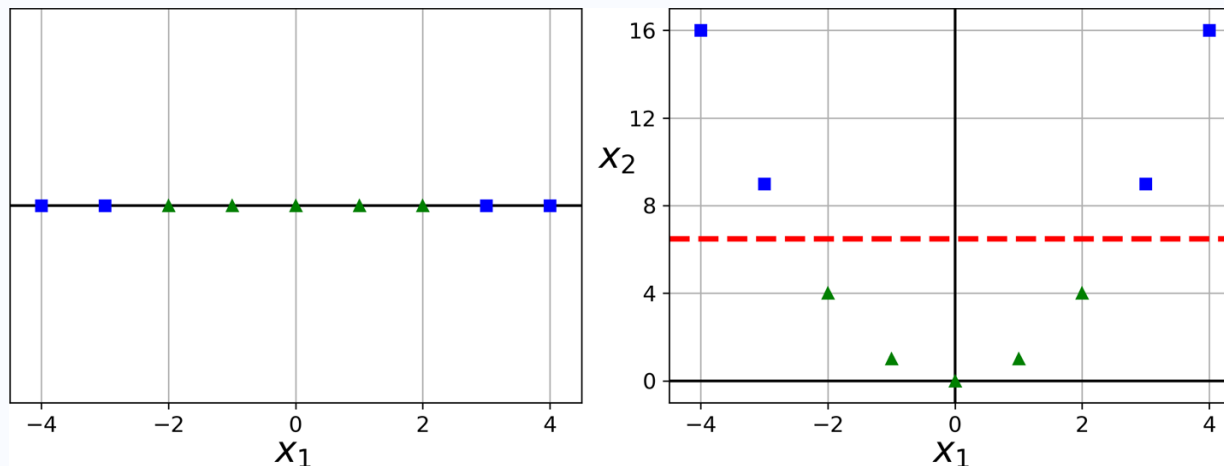
$$\text{subject to } t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \zeta^{(i)} \text{ and } \zeta^{(i)} \geq 0 \text{ for } i = 1, 2, \dots, m$$

Quadratic Programming

- Both hard margin and soft margin problems are convex optimization problems with linear constraints
- Such problems are known as Quadratic Programming (QP) problems
- Many off-the-shelf solvers are available to solve QP problems by using a variety of techniques that are outside the scope of this module
- We will use readily available packages in our labs to work with SVMs

Nonlinear SVM

- Very common to encounter datasets that are not linearly separable
- You can add more features – in some cases, this results in a linearly separable dataset



Adding a second feature $x_2 = (x_1)^2$ makes the resulting 2D dataset linearly separable

- Polynomial features: too low degree = underfitting, too high degree = very slow computation time

Kernel Trick

- *Kernel trick* can lead to the same result as adding polynomial features
 - The kernel trick lets SVMs **act as if** the data has been lifted into a higher-dimensional space — **without ever doing the lifting**
 - In Machine Learning, a *kernel* is a function capable of computing the dot product $\phi(\mathbf{a})^\top \phi(\mathbf{b})$, based only on the original vectors $\mathbf{a}$ and $\mathbf{b}$, without having to compute (or even to know about) the transformation ϕ .
 - E.g. The function $K(\mathbf{a}, \mathbf{b}) = (\mathbf{a}^\top \mathbf{b})^2$ is a second-degree polynomial kernel. Some other kernels:

Linear: $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$

Polynomial: $K(\mathbf{a}, \mathbf{b}) = \gamma \mathbf{a}^\top \mathbf{b} + r)^d$

Gaussian RBF: $K(\mathbf{a}, \mathbf{b}) = \exp(-\gamma \|\mathbf{a} - \mathbf{b}\|^2)$

Sigmoid: $K(\mathbf{a}, \mathbf{b}) = \tanh(\gamma \mathbf{a}^\top \mathbf{b} + r)$

A kernel reproduces the feature-space dot product

1 Compute in the original space

$$\mathbf{a} = (1, 2) \quad \mathbf{b} = (3, 1)$$

$$\mathbf{a}^\top \mathbf{b} = 1 \times 3 + 2 \times 1$$

$$\mathbf{a}^\top \mathbf{b} = 5$$

$$K(\mathbf{a}, \mathbf{b}) = (\mathbf{a}^\top \mathbf{b})^2$$

$$K(\mathbf{a}, \mathbf{b}) = 25$$

2 Verify in the feature space

$$\phi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

$$\phi(\mathbf{a}) = (1, 2\sqrt{2}, 4)$$

$$\phi(\mathbf{b}) = (9, 3\sqrt{2}, 1)$$

$$\phi(\mathbf{a})^\top \phi(\mathbf{b})$$

$$= 1 \times 9 + (2\sqrt{2})(3\sqrt{2}) + 4 \times 1$$

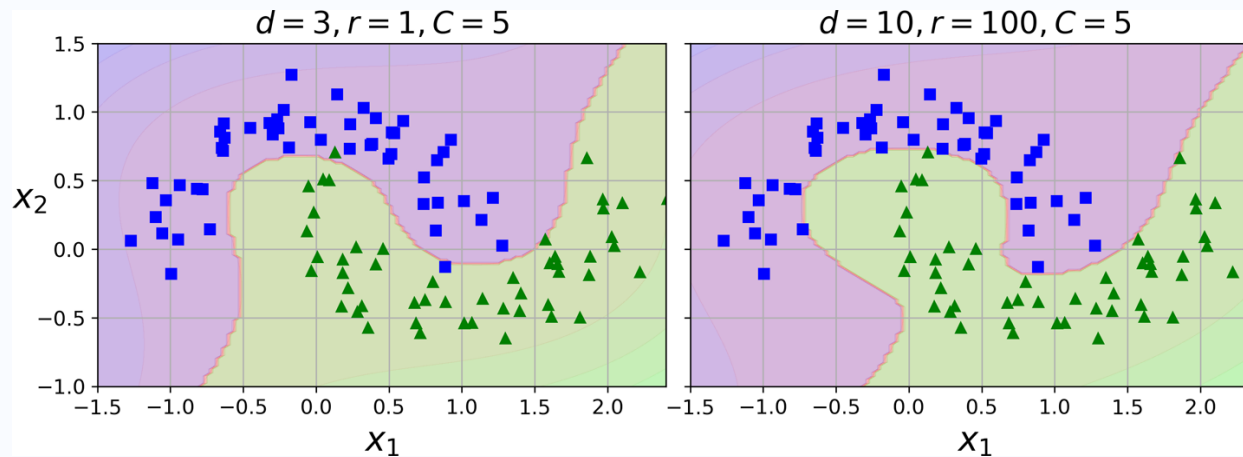
$$= 9 + 12 + 4 = 25$$

SAME RESULT

$$K(\mathbf{a}, \mathbf{b}) = \phi(\mathbf{a})^\top \phi(\mathbf{b}) = 25$$

The shortcut avoids explicitly constructing $\phi(\mathbf{a})$ and $\phi(\mathbf{b})$.

Kernal trick II



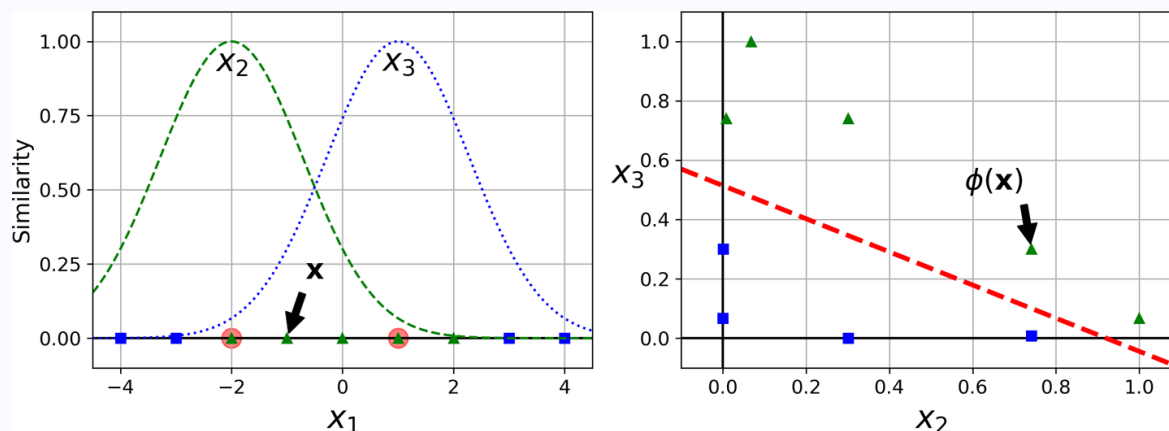
- d = degree
- C = margin hyperparameter
- r = how much the model is influenced by high-degree polynomials vs low-degree ones

Similarity Features – Gaussian RBF

- Another method to tackle nonlinear datasets – use *similarity function*, which measures how much each instance resembles a particular *landmark*.
- Gaussian radial basis function:

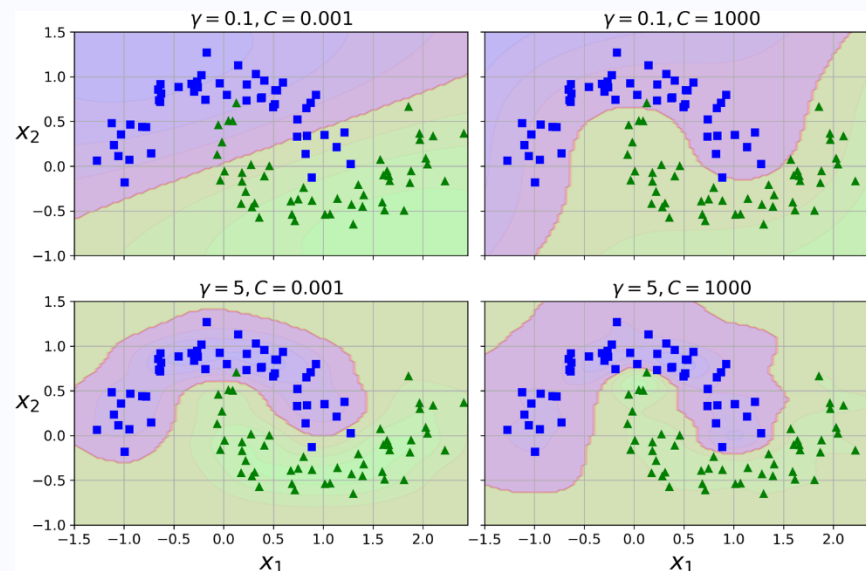
$$\phi_{\gamma}(\mathbf{x}, \ell) = \exp(-\gamma \|\mathbf{x} - \ell\|^2)$$

- Let's set two landmarks at $x_1 = -2$ and $x_1 = 1$ and $\gamma = 0.3$
 - Creates 2 new features for the point $x_1 = -1 \rightarrow x_2 = \exp(-0.3 \times 1^2) \approx 0.74$ and $x_3 = \exp(-0.3 \times 2^2) \approx 0.30$



Similarity Features – Gaussian RBF

- How to set landmarks?
 - Each data instance can be a landmark $\rightarrow$ this creates many dimensions and increases the chance of transforming the training set to a linearly separable one
 - It may be computationally expensive
- You can use the kernel trick here. Use Gaussian RBF kernel!



Summary

- In this lecture, we looked at
 - Logistic Regression
 - Softmax Regression
 - Linear SVMs
 - Nonlinear SVMs – kernel trick
- You will take a closer look at these techniques by implementing them during the labs.
- Note: Labs start this week, Friday, 4-6 pm, for the next 7 weeks

Next Lecture: Data Preparation, Validation and Model Comparison

Questions?